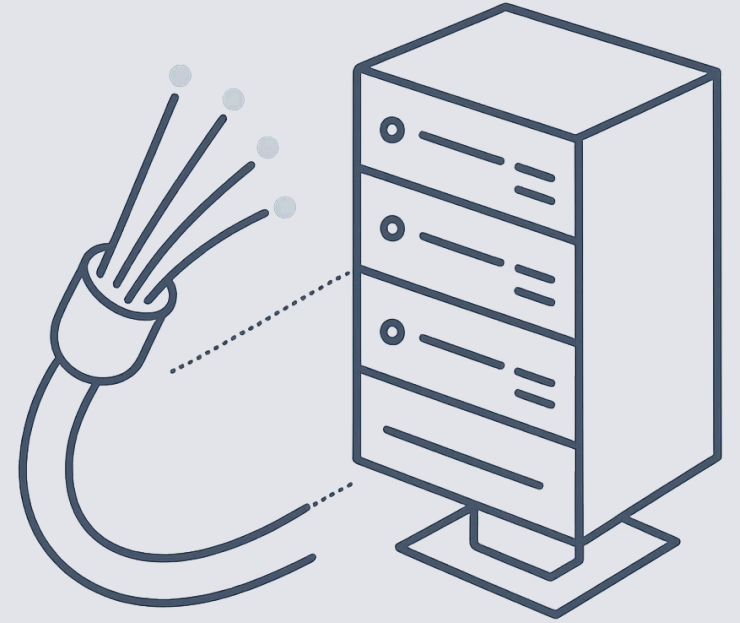


# WebSocket avec Spring Boot



Au programme

# Ce que vous allez apprendre

01

---

## Le protocole WebSocket

Comprendre les fondamentaux et les avantages face au HTTP classique.

02

---

## Configuration Spring

Mettre en place un serveur WebSocket avec Spring Boot.

03

---

## STOMP & Broker

Structurer les messages et router les communications.

04

---

## SockJS & Sécurité

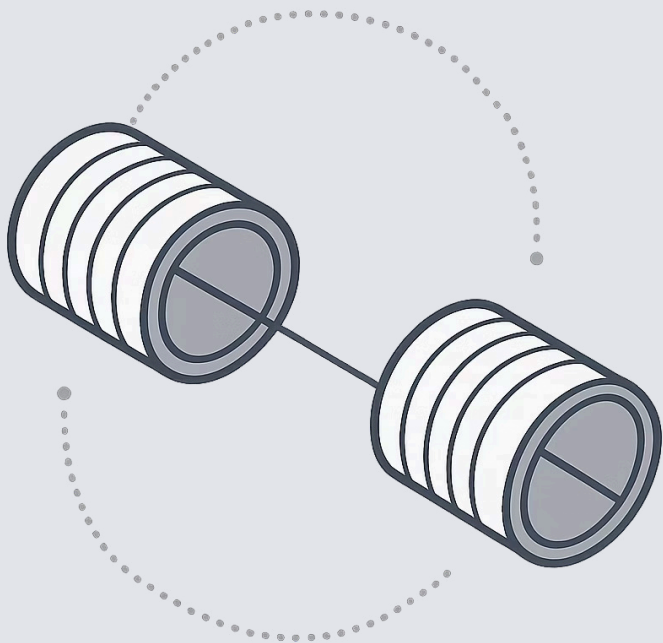
Assurer compatibilité et contrôle d'accès en production.

05

---

## Travaux pratiques

Implémenter une messagerie temps réel de bout en bout.



CHAPITRE 1

# Le protocole WebSocket

# HTTP vs WebSocket : le problème fondamental

## HTTP classique (requête/réponse)

Le client **demande**, le serveur **répond**, la connexion se ferme.  
Pour simuler du temps réel, il faut interroger le serveur en boucle (*polling*) — coûteux et inefficace.

⚠ Chaque requête HTTP ouvre et ferme une connexion TCP. Le overhead est important.

## WebSocket (canal bidirectionnel)

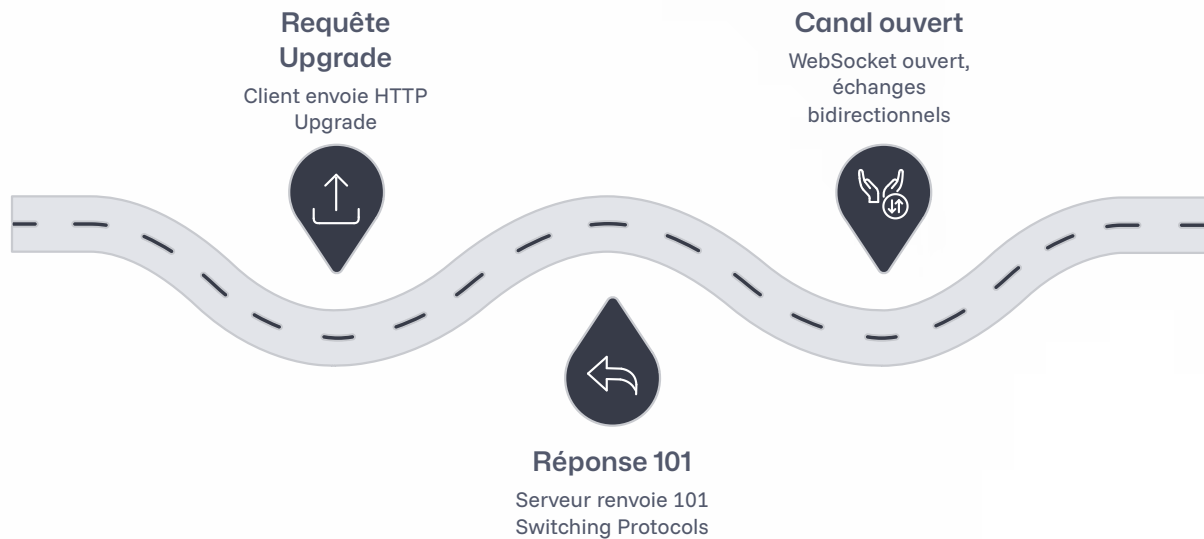
Une seule connexion TCP est établie et **maintenue ouverte**.  
Client et serveur peuvent s'envoyer des messages à tout moment, sans overhead de reconnexion.

✅ Latence très faible, communication full-duplex en temps réel.



# La poignée de main WebSocket

WebSocket démarre par une **requête HTTP Upgrade**. Une fois acceptée, le protocole bascule sur un canal TCP persistant.



Après le handshake, ni entêtes HTTP ni cycles requête/réponse : seulement des **frames** légères dans les deux sens.



# Avantages clés de WebSocket

## Faible latence

Pas de reconnexion à chaque message. Idéal pour le temps réel.

## Full-duplex

Serveur et client parlent simultanément sur le même canal.

## Moins de bande passante

Frames légères, pas d'entêtes HTTP répétés à chaque échange.

## Push serveur natif

Le serveur pousse les données sans que le client ne les demande.

# Cas d'usage typiques



## Chat temps réel

Messagerie instantanée, support client en ligne.



## Cours financiers

Mise à jour des prix en temps réel sans polling.



## Jeux multijoueur

Synchronisation d'état entre joueurs en temps réel.



## Monitoring live

Tableaux de bord avec métriques actualisées en continu.

## CHAPITRE 2

# L'analogie du système postal

Pour comprendre les rôles de **WebSocket**, **STOMP**, du **Broker** et de **SockJS** — imaginons un bureau de poste.



# L'analogie expliquée



## **WebSocket = La route**

L'infrastructure physique qui relie deux points. Elle transporte les données, mais ne dit rien sur leur format ni leur destination finale. Sans route, rien ne circule.



## **STOMP = L'enveloppe et l'adresse**

Le format standardisé d'un message : à qui il est destiné, de qui il vient, quel est son contenu. STOMP structure ce qui circule sur la route WebSocket.



## **Broker = Le bureau de tri postal**

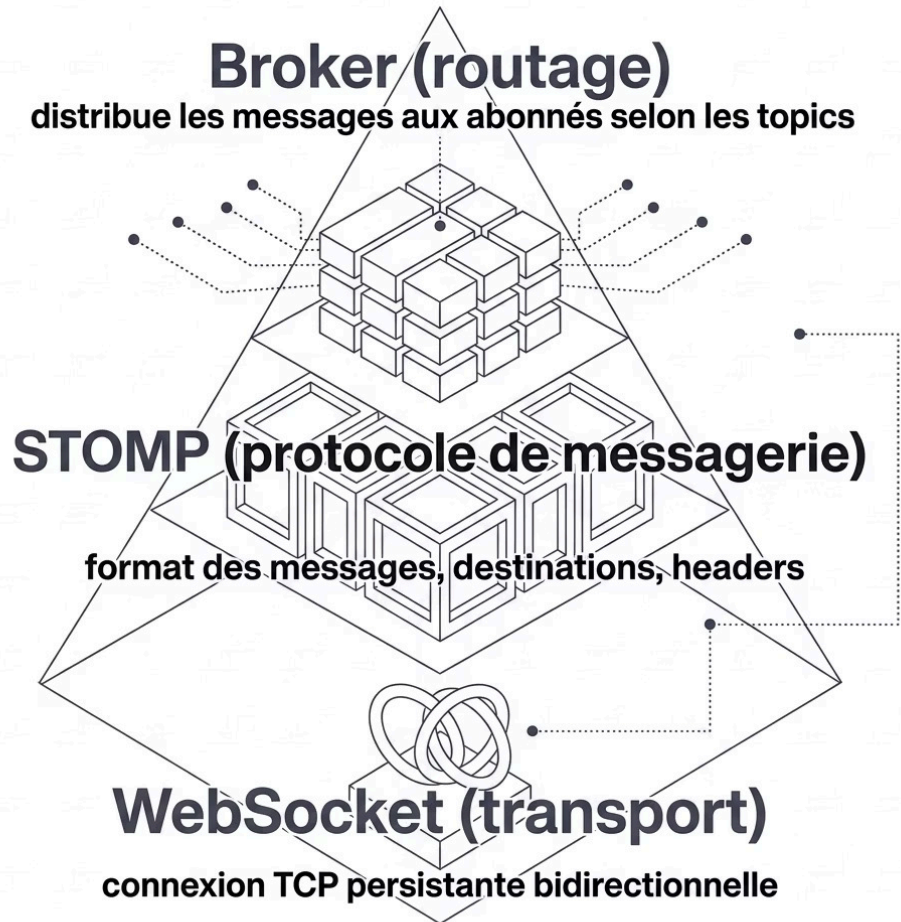
Il reçoit les enveloppes et les distribue aux bons destinataires selon leur adresse (topic). Il gère les abonnements et sait qui veut recevoir quoi.



## **SockJS = Le service de remplacement**

Si la route principale (WebSocket) est coupée, SockJS emprunte d'autres chemins (long-polling, XHR) pour que le courrier arrive quand même à destination.

# WebSocket, STOMP, Broker : qui fait quoi ?



## Les trois couches distinctes

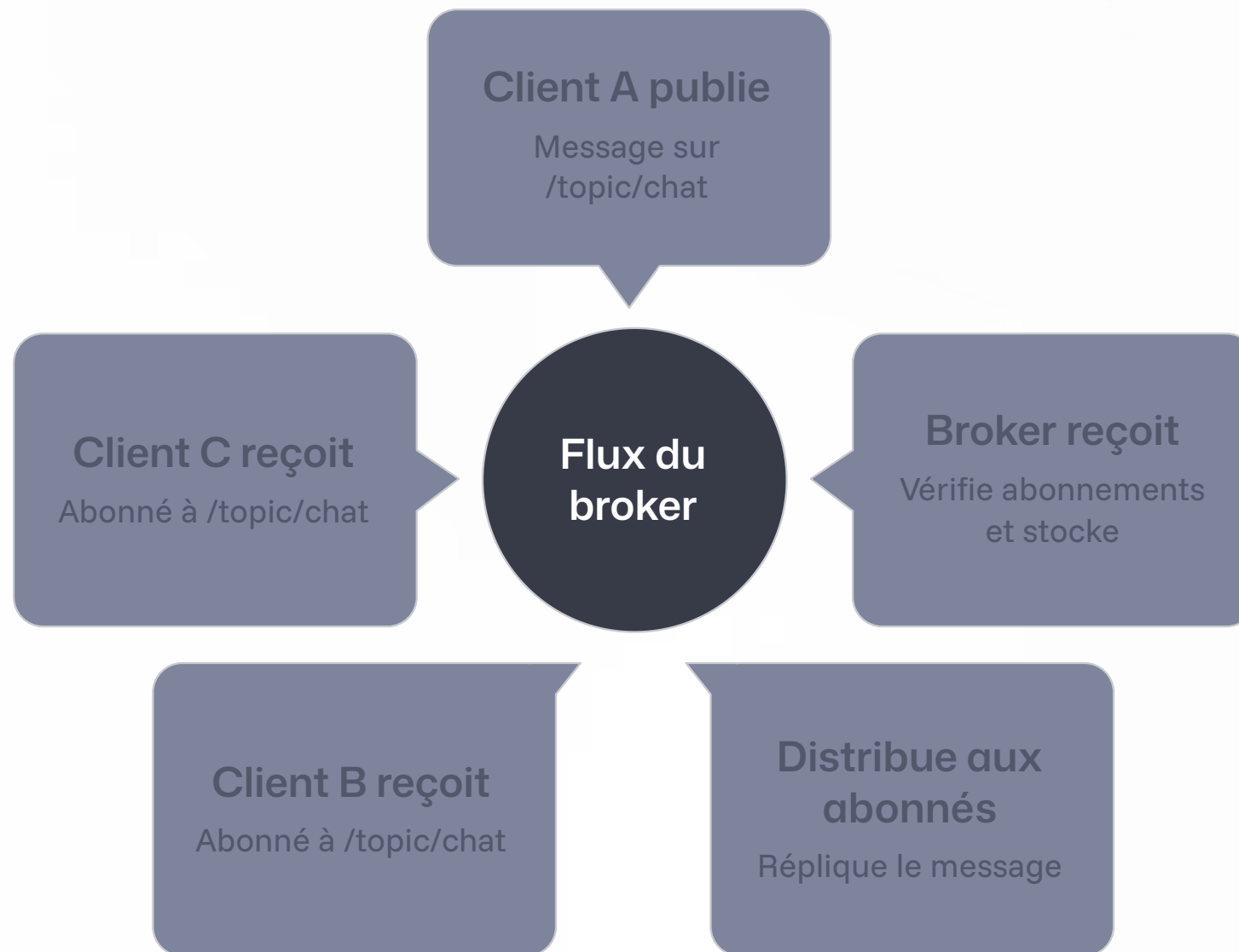
**WebSocket** est uniquement une couche de *transport*. Il ne sait pas ce que contiennent les données.

**STOMP** (Simple Text Oriented Messaging Protocol) ajoute une sémantique : `SEND`, `SUBSCRIBE`, `MESSAGE`... Il structure la conversation.

Le **broker** est le cerveau de la distribution : il maintient la liste des abonnements et route chaque message vers les bons clients.

**i** STOMP peut fonctionner sur WebSocket, mais aussi sur d'autres transports. Le broker est indépendant de la connexion cliente.

# Le rôle central du Broker



Le broker maintient un registre des **abonnements**. Quand un message arrive sur un topic, il le recopie et l'envoie à chaque abonné. Sans broker, chaque client devrait connaître tous les autres clients.

CHAPITRE 3

# Configuration Spring WebSocket





# Dépendances Maven

Ajoutez le starter Spring WebSocket dans votre `pom.xml` :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

- ❗ Spring Boot auto-configuré une grande partie de l'infrastructure. La dépendance inclut Spring Messaging, nécessaire pour STOMP et le broker.

# La classe de configuration WebSocket

La configuration centrale se fait via `@EnableWebSocketMessageBroker` :

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic", "/queue");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")
            .withSockJS();
    }
}
```

# Décrypter la configuration

```
enableSimpleBroker("/topic",  
"/queue")
```

Active le broker en mémoire. Les messages vers `/topic/...` sont diffusés à tous les abonnés (broadcast). `/queue/...` cible un utilisateur spécifique (point-à-point).

```
setApplicationDestinationPr  
efixes("/app")
```

Les messages envoyés vers `/app/...` sont routés vers les méthodes `@RequestMapping` de vos contrôleurs Spring avant traitement.

```
addEndpoint("/ws").withSock  
JS()
```

Expose le point d'entrée WebSocket à l'URL `/ws`. `.withSockJS()` active le fallback automatique pour les navigateurs incompatibles.

# Le contrôleur de messages

Côté serveur, un `@Controller` traite les messages entrants :

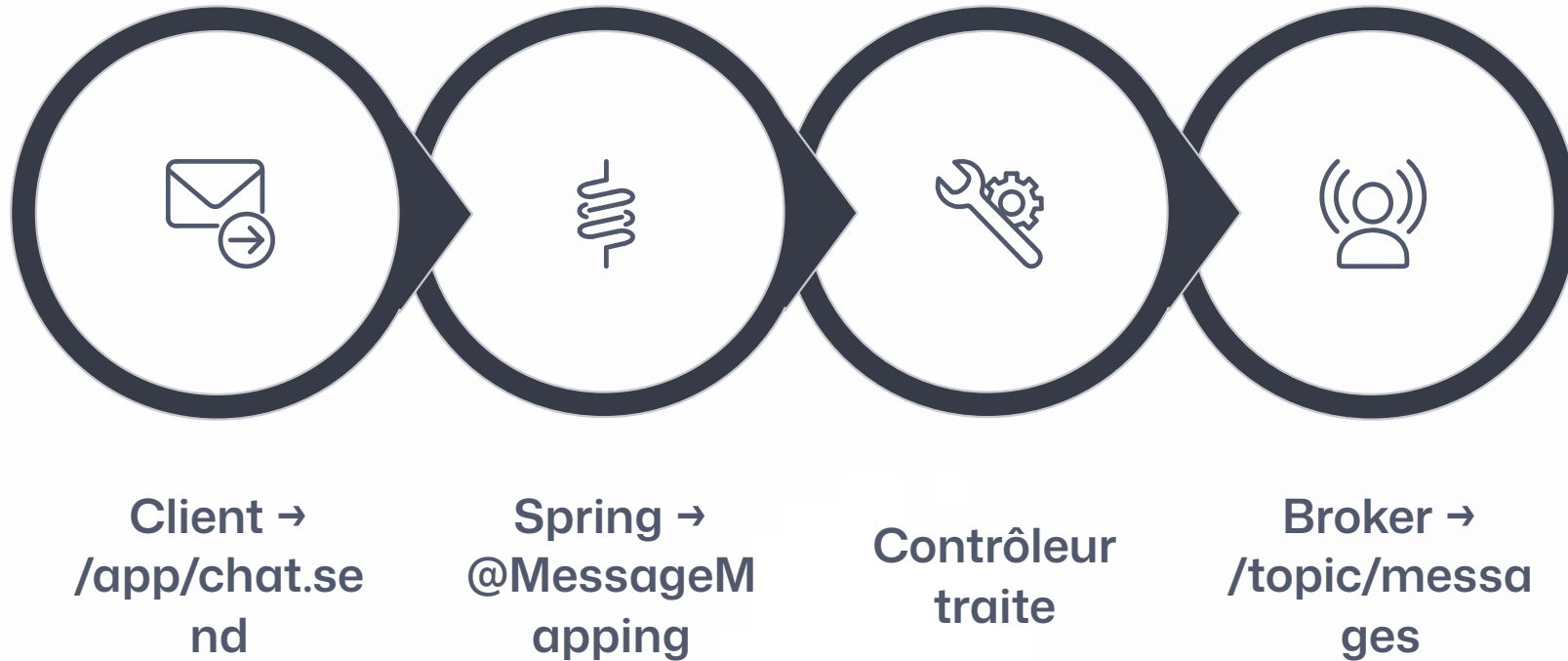
```
@Controller
public class ChatController {

    @RequestMapping("/chat.send")
    @SendTo("/topic/messages")
    public Message sendMessage(Message message) {
        return message; // diffusé à tous les abonnés
    }

    @RequestMapping("/chat.join")
    @SendTo("/topic/messages")
    public Message addUser(Message message, SimpMessageHeaderAccessor accessor) {
        accessor.getSessionAttributes().put("username", message.getSender());
        return message;
    }
}
```

❏ `@RequestMapping` capte les messages arrivant sur `/app/chat.send`. `@SendTo` diffuse la réponse sur le topic indiqué.

# Flux complet d'un message



Le préfixe `/app` déclenche le passage par le contrôleur. Le préfixe `/topic` pointe directement vers le broker pour la diffusion.



#### CHAPITRE 4

# STOMP : le protocole de messagerie

# STOMP en détail

## Qu'est-ce que STOMP ?

**Simple Text Oriented Messaging Protocol** est un protocole de messagerie léger, basé sur du texte, inspiré du modèle HTTP.

Il définit des **frames** (trames) avec une commande, des headers et un body :

```
SEND
destination:/app/chat.send
content-type:application/json

{"sender":"Alice","content":"Bonjour"}
^@
```

## Commandes STOMP principales

### CONNECT

Initier une session  
STOMP

### SUBSCRIBE

S'abonner à un topic

### SEND

Envoyer un message

### DISCONNECT

Clore la session

# Client JavaScript STOMP

Côté navigateur, on utilise la librairie `@stomp/stompjs` :

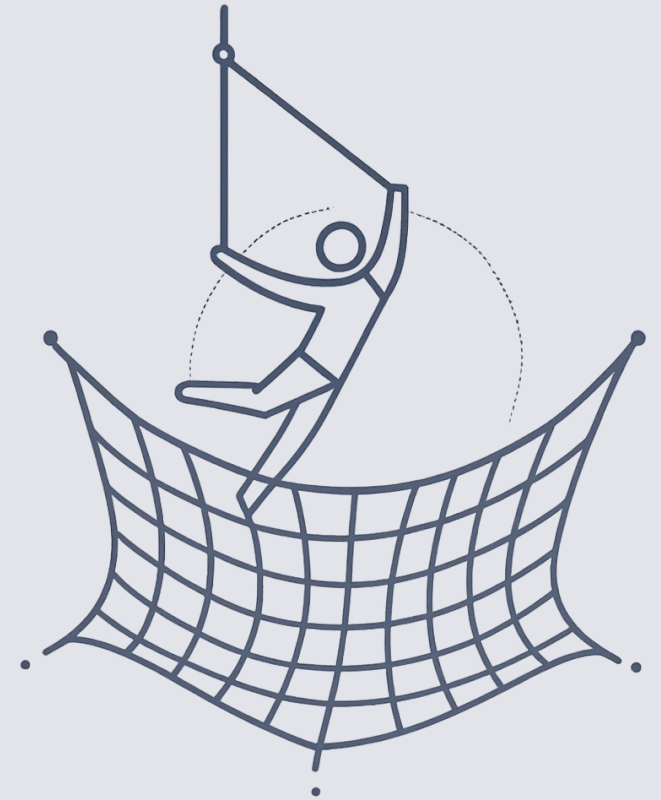
```
const client = new Client({
  brokerURL: 'ws://localhost:8080/ws',
  onConnect: () => {
    // S'abonner à un topic
    client.subscribe('/topic/messages', (message) => {
      const body = JSON.parse(message.body);
      console.log('Reçu :', body);
    });

    // Envoyer un message
    client.publish({
      destination: '/app/chat.send',
      body: JSON.stringify({ sender: 'Alice', content: 'Bonjour !' })
    });
  }
});

client.activate();
```



# SockJS : le filet de sécurité



# Pourquoi SockJS ?

## Le problème

WebSocket peut être **bloqué** par certains proxies d'entreprise, pare-feux, ou anciens navigateurs. Dans ces environnements, la connexion échoue silencieusement.

 En production, vous ne contrôlez pas l'infrastructure réseau de vos utilisateurs.

## La solution SockJS

SockJS tente d'abord une connexion WebSocket native. En cas d'échec, il bascule **automatiquement** sur des alternatives :

- HTTP Long-Polling
- HTTP Streaming
- XHR avec multipart

L'API reste identique pour le développeur.

# SockJS côté client

Avec SockJS activé côté serveur (`.withSockJS()`), le client utilise :

```
import SockJS from 'sockjs-client';
import { Client } from '@stomp/stompjs';

const client = new Client({
  websocketFactory: () => new SockJS('http://localhost:8080/ws'),
  onConnect: () => {
    client.subscribe('/topic/messages', (msg) => {
      console.log(JSON.parse(msg.body));
    });
  }
});

client.activate();
```

✅ Le seul changement : `websocketFactory` remplace `brokerURL`. Le reste du code est identique.

## CHAPITRE 6

# Sécurité des WebSockets



# Sécuriser les WebSockets avec Spring Security

## Configuration Spring Security 6+

@Configuration

```
public class WebSocketSecurityConfig {
```

@Bean

```
public AuthorizationManager<Message<?>>  
messageAuthorizationManager(  
    MessageMatcherDelegatingAuthorizationManager.Builder messages) {  
    messages  
        .simpDestMatchers("/app/**").authenticated()  
        .simpSubscribeDestMatchers("/topic/**").authenticated()  
    }  
    .anyMessage().denyAll();  
    return messages.build();  
}
```



### Dépréciation :

`AbstractSecurityWebSocketMessageBrokerConfigurer` est dépréciée depuis Spring Security 6. La protection CSRF pour WebSocket est désormais gérée via `CsrfTokenHandshakeInterceptor` ou en la désactivant dans le `SecurityFilterChain`.

## Points clés

### → Authentification au handshake

Spring Security valide le token/session HTTP lors de la connexion initiale.

### → Autorisation par destination

Contrôler qui peut envoyer (`/app/**`) et s'abonner (`/topic/**`).

### → CSRF et WebSocket

Depuis Spring Security 6+, le CSRF pour WebSocket est géré via `CsrfTokenHandshakeInterceptor` ou désactivé explicitement dans le `SecurityFilterChain`, en remplacement de `sameOriginDisabled()`.

# Bonnes pratiques de sécurité



## WSS (TLS)

Toujours utiliser `wss://` en production. Jamais `ws://` en clair.



## Authentification JWT

Passer le token dans les headers STOMP CONNECT ou via un interceptor.



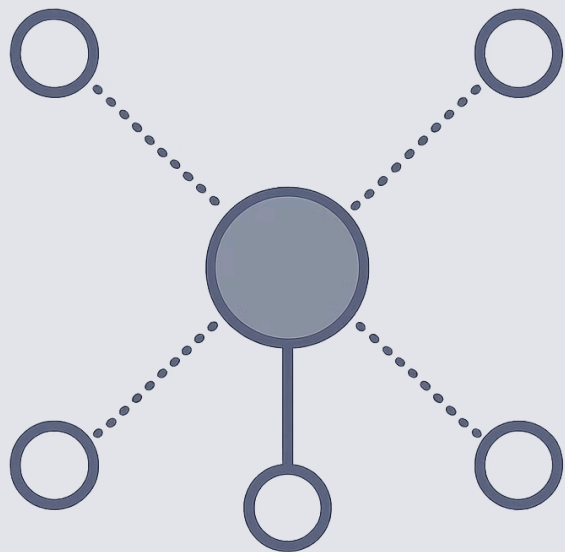
## Validation des messages

Valider chaque message reçu côté serveur, comme pour une API REST.



## Limiter les origines

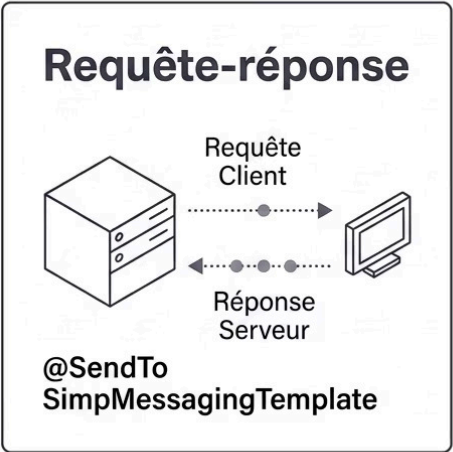
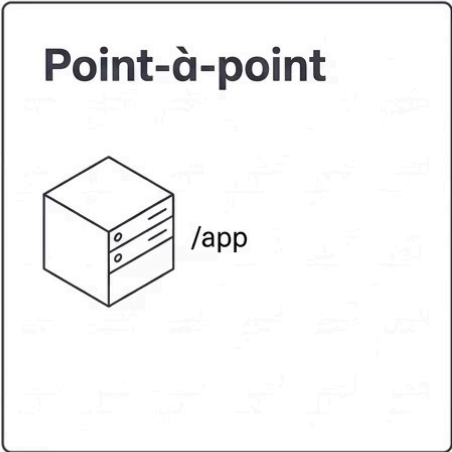
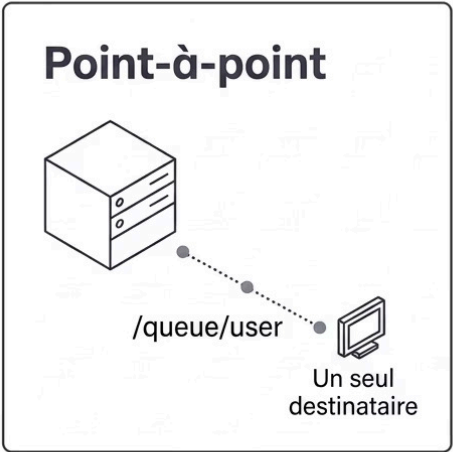
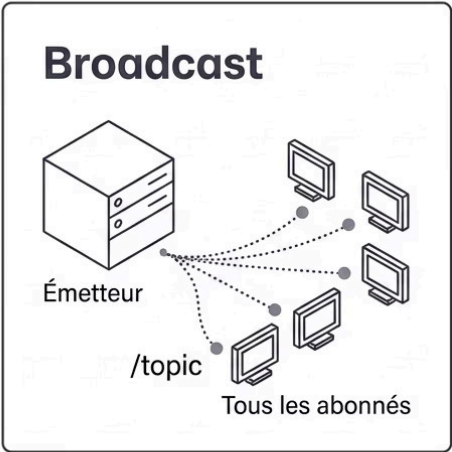
Configurer `setAllowedOrigins()` pour restreindre les connexions cross-origin.



## CHAPITRE 7

# Patterns de communication temps réel

# Les trois patterns essentiels



## Choisir le bon pattern

**Broadcast (/topic)** : notifications globales, chat de groupe, mises à jour en direct pour tous.

**Point-à-point (/queue)** : messages privés, résultats d'une opération propres à un utilisateur.

**Requête-réponse** : actions métier nécessitant un traitement serveur avant réponse.



# Envoyer un message à un utilisateur spécifique

Spring facilite l'envoi ciblé via `SimpMessagingTemplate` :

```
@Autowired
private SimpMessagingTemplate messagingTemplate;

@MessageMapping("/private.message")
public void sendPrivateMessage(Message message) {
    // Envoie uniquement à l'utilisateur ciblé
    messagingTemplate.convertAndSendToUser(
        message.getRecipient(), // username
        "/queue/private",      // destination
        message                 // payload
    );
}
```

 Le client s'abonne à `/user/queue/private`. Spring résout automatiquement la session de l'utilisateur connecté.

# Broker externe : RabbitMQ / ActiveMQ

## Broker en mémoire vs externe

### SimpleBroker (dev)

Intégré à Spring, zéro configuration. Limité à une seule instance JVM.

### Broker externe (prod)

RabbitMQ ou ActiveMQ pour la scalabilité, la persistance et le clustering.

## Configuration RabbitMQ

```
@Override
public void configureMessageBroker(
    MessageBrokerRegistry config) {
    config.enableStompBrokerRelay("/topic", "/queue")
        .setRelayHost("localhost")
        .setRelayPort(61613)
        .setClientLogin("guest")
        .setClientPasscode("guest");
    config.setApplicationDestinationPrefixes("/app");
}
```

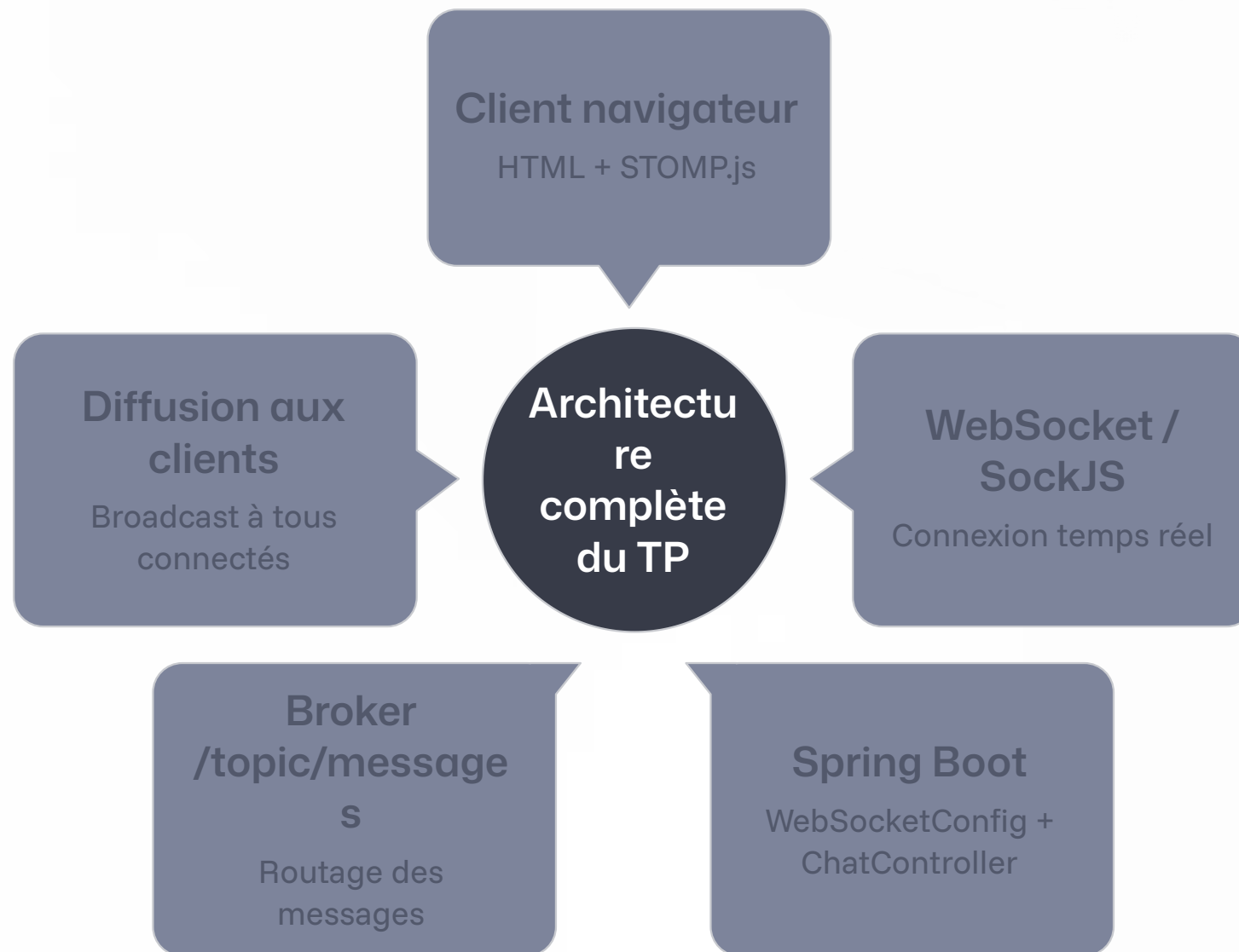
Remplacez `enableSimpleBroker` par `enableStompBrokerRelay` pour pointer vers un broker externe.

TRAVAUX PRATIQUES

# TP – Chat temps réel avec Spring WebSocket



# Architecture du TP



Vous allez construire un chat multi-utilisateurs complet : connexion, envoi de messages, notification d'arrivée, le tout en temps réel sans aucun polling.

# Étape 1 – Projet et configuration

## 1 Créer le projet

Générer un projet Spring Boot via [start.spring.io](https://start.spring.io) avec les dépendances Spring WebSocket et Spring Web.

## 2 Classe WebSocketConfig

Implémenter `WebSocketMessageBrokerConfigurer`, activer le broker sur `/topic`, définir le préfixe `/app`, exposer l'endpoint `/ws` avec SockJS.

## 3 Modèle de message

Créer un POJO `ChatMessage` avec les champs `type` (CHAT/JOIN/LEAVE), `sender` et `content`.

# Étape 2 – Contrôleur et frontend

## Le contrôleur

```
@Controller
public class ChatController {

    @RequestMapping("/chat.send")
    @SendTo("/topic/public")
    public ChatMessage send(ChatMessage msg) {
        return msg;
    }

    @RequestMapping("/chat.addUser")
    @SendTo("/topic/public")
    public ChatMessage addUser(ChatMessage msg,
        SimpMessageHeaderAccessor sha) {
        sha.getSessionAttributes()
            .put("username", msg.getSender());
        return msg;
    }
}
```

## Ce que vous allez tester

- Connexion de plusieurs onglets simultanément
- Envoi d'un message visible par tous
- Notification de connexion/déconnexion
- Observer les frames WebSocket dans les DevTools

# Étape 3 – Gérer les événements de session

Détecter les déconnexions via les événements Spring :

```
@Component
public class WebSocketEventListener {

    @Autowired
    private SimpMessageSendingOperations messagingTemplate;

    @EventListener
    public void handleWebSocketDisconnectListener(
        SessionDisconnectEvent event) {
        StompHeaderAccessor sha =
            StompHeaderAccessor.wrap(event.getMessage());
        String username = (String) sha.getSessionAttributes()
            .get("username");
        if (username != null) {
            ChatMessage msg = new ChatMessage(Type.LEAVE, username, null);
            messagingTemplate.convertAndSend("/topic/public", msg);
        }
    }
}
```

✅ Spring publie `SessionDisconnectEvent` automatiquement. Plus besoin de gérer manuellement les timeouts de connexion.

# Points de vérification du TP

1

## Connexion établie

Le navigateur se connecte à `/ws` et affiche le nom d'utilisateur dans la liste.

2

## Broadcast fonctionnel

Un message envoyé depuis un onglet apparaît immédiatement dans tous les autres.

3

## Déconnexion gérée

La fermeture d'un onglet déclenche une notification "a quitté le chat" pour les autres.

4

## Fallback SockJS actif

En désactivant WebSocket dans le navigateur, la connexion se maintient via long-polling.



# Pour aller plus loin



## JWT dans STOMP

Implémenter un `ChannelInterceptor` pour valider un token JWT à chaque CONNECT.



## RabbitMQ en production

Remplacer le SimpleBroker par un broker STOMP externe pour scaler horizontalement.



## Salons privés

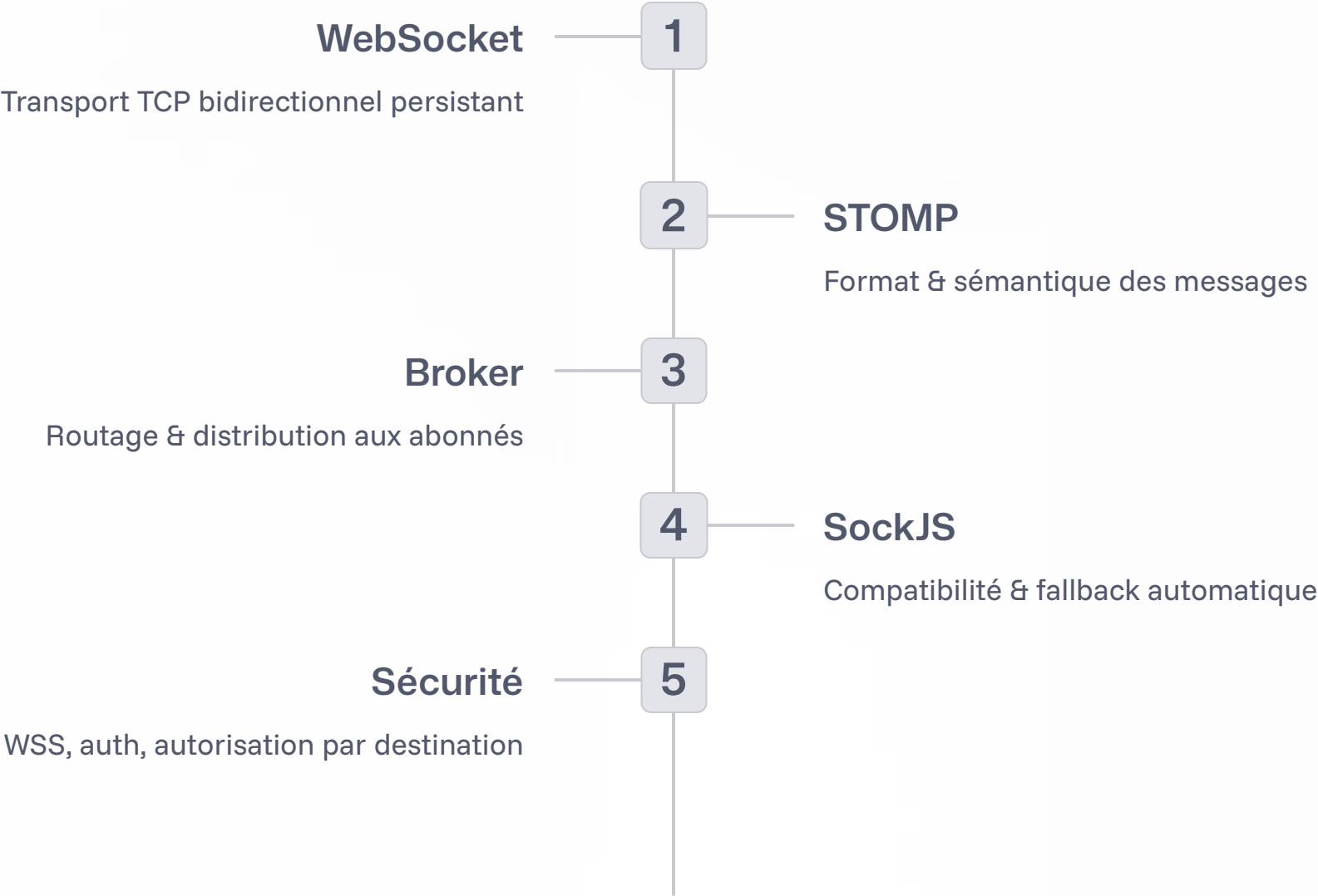
Utiliser `convertAndSendToUser` pour implémenter des messages directs entre utilisateurs.



## Tests de charge

Utiliser Gatling ou Artillery pour simuler des milliers de connexions WebSocket concurrentes.

# Récapitulatif



# Questions & échanges

*"La meilleure façon d'apprendre un protocole, c'est d'en observer les trames en action."*

— Ouvrez les DevTools → Network → WS pour voir vos frames STOMP en direct.

## Docs officielles

[docs.spring.io/spring-framework/reference/web/websocket.html](https://docs.spring.io/spring-framework/reference/web/websocket.html)

## Code du TP

Disponible dans le dépôt Git partagé en formation.

## Prochaine étape

Intégration WebSocket dans un projet React ou Angular avec authentification JWT.